

SentinelID — Project Status Report

Phase 2 (Backend Integration) + Phase 3 (Frontend Wiring) in progress

Snapshot — update as modules are completed

Overview

Phase 2 wraps the three AI modules (P1's OCR/MRZ extraction, P2's tampering detection, P4's face verification) into a single FastAPI server, exposed through one combined endpoint, `POST /verify`. This has been tested end-to-end with a real ID document and selfie, including PDF uploads. Phase 3 work has now started: the frontend's upload screen and main app logic have been rewired to call the real backend instead of mock data.

Module Status

Module	Status	Notes
FastAPI server	Working	5 endpoints live: <code>/</code> , <code>/ocr/extract</code> , <code>/tamper/check</code> , <code>/face/verify</code> , <code>/verify</code>
PDF upload support	Working	Auto-converts first page of a PDF to PNG before running any module
OCR / MRZ (P1)	Working	Raw text confirmed on a real ID card. MRZ correctly null for non-passport documents (expected, not a bug)
Tampering detection (P2 - model)	Placeholder	Running ELA + missing-EXIF heuristic only. P2's trained model not yet integrated — training in progress
Face verification (P4)	Working	DeepFace/ArcFace comparison confirmed for both match and no-match cases
Risk-score fusion	Working (client-side)	<code>computeRisk()</code> runs in the frontend on the real API response — no backend change needed for this to work
Frontend: API layer	Working	<code>verifyDocument.js</code> sends document + selfie to <code>/verify</code> and returns the JSON result
Frontend: Upload screen	Working	Rebuilt to collect both a document and a selfie file, with a submit button
Frontend: App wiring	Working	<code>App.jsx</code> now calls the real backend via <code>async/await</code> , with error handling back to the upload screen
Frontend: Results display	Not yet verified	<code>ResultsDashboard.jsx</code> / <code>ProcessingScreen.jsx</code> not yet checked against the real API response shape

What's confirmed working end-to-end

- Backend server starts and all five endpoints appear on /docs
- Uploading a PDF document auto-converts and runs through all three modules without error
- OCR returns readable raw text from a real ID card; MRZ correctly returns null for non-passport documents
- Tampering check runs and returns a score + verdict (placeholder logic — see gaps below)
- Face verification correctly returns both match and no-match results
- Frontend upload screen collects both required files and only enables submit once both are present
- App.jsx successfully swapped from mock data to a real, awaited backend call, with basic error handling

Known gaps / open items

1. Tampering model swap (blocked on P2)

P2 is still training the real tampering-detection model. The backend currently uses a placeholder (Error Level Analysis + missing-EXIF check), which flags almost any re-saved or screenshotted image as "SUSPICIOUS" — expect false positives until the real model is in. When ready, P2 provides the trained model file and rewrites the inside of `tamper_module.py` (same function name/signature), which fully replaces the placeholder. No changes needed to `main.py`.

2. Results display not yet verified against real data

The mock data included a field (`tamper.ela_output_path`) that the real backend does not currently return. `ResultsDashboard.jsx` and `ProcessingScreen.jsx` were built and presumably tested against the mock shape, but haven't been checked against the real API response yet — worth reviewing before considering the frontend integration complete.

3. Not yet handled

- Video-based tampering checks (optional stretch from P2's original notebook)
- Deploying the backend anywhere other than localhost
- Broader testing: passport-style documents (to confirm MRZ parsing), blurry/low-quality images, mismatched selfies

Next Steps

Owner	Next action
You (P3)	Share <code>ResultsDashboard.jsx</code> and <code>ProcessingScreen.jsx</code> so they can be checked/updated against the real API response shape; run a full end-to-end test in the browser with the Network tab open
P1	Confirm whether client-side <code>computeRisk()</code> is the final approach, or whether risk fusion should move server-side later; continue database work
P2	Finish training the tampering model; hand off the model file and describe its output format when ready
P4	Available to help debug the P2 model integration, or start on the identity-graph stretch goal

Generated as a snapshot of current progress — update as modules are completed.